IMPLEMENTATION-FIRST WINDOWS 11 PRO GUIDE

# Windows Wired Network Performance & Resilience Runbook

10
GbE SCALE

Driver baseline • low-latency configuration • symmetric multi-gigabit validation • deterministic host failover

**EVIDENCE BOUNDARY**

Field-tested at 2 Gbps up / 2 Gbps down

Scaling method to 5/5 and 10/10 is documentation-backed and Amy-untested

FIELD VERIFIED — AMY'S 2/2 GBPS RECORD

AMY-UNTESTED — 5/5 + 10/10 APPENDIX

START HERE

# Choose the correct path

PROVEN BASELINE

Do not tune a healthy link before establishing what is actually failing.

## 1 VALIDATE

Use when the link feels slow but no configuration change is justified yet.

- Capture the baseline package.
- Pass the local gate.
- Measure idle, one direction, then both.
- Stop if the evidence already meets the target.

## 2 REPAIR

Use for link drops, wrong negotiation, driver regression, or counter errors.

- Restore a known-good driver.
- Return advanced properties to defaults.
- Replace one physical component at a time.
- Revalidate locally before WAN testing.

## 3 EXPERIMENT

Use only for a named symptom and an A/B measurement.

- Change one property.
- Repeat the same workload three times.
- Keep only a repeatable improvement.
- Roll back neutral or worse results.

## 4 FAILOVER

Use to prefer a primary path and test recovery to a secondary path.

- Define failure domains.
- Set interface metrics.
- Expect existing TCP sessions to reset.
- Do not call one shared ISP redundant.

## Evidence labels used throughout

PROVEN BASELINE - authoritative documentation and safe default behavior.

FIELD VERIFIED - observed in Amy's recorded 2/2 Gbps environment.

CONDITIONAL CHANGE - justified only by a named symptom or deployment need.

CONTROLLED EXPERIMENT - one-change A/B test; retain only after measured improvement.

AVOID - unsafe bulk changes, unmeasured folklore, or evidence-conflicting advice.

## Release integrity

Use a PDF from the official tagged release and match it to the same-version Markdown companion.

Verify the file against outputs/SHA256SUMS.txt before relying on a downloaded copy.

Modified, unversioned, or checksum-mismatched copies are outside the supported release.

**START**

Run pages 4-5 before any change.

**RESTART**

None for reading or baseline capture.

**PASS**

The chosen path matches the observed symptom.

**FAIL**

The path assumes a cause without evidence.

**ROLLBACK**

Return to VALIDATE and capture evidence.

FIELD RECORD

# What Amy's 2/2 Gbps evidence proves

FIELD VERIFIED

A bounded result, not a promise about every server, route, adapter, or future date.

**1.9**
Gbps upload and approximately 1.9 Gbps download in one simultaneous run.

**3.8**
Gbps simultaneous aggregate application throughput in that run.

**3.9**
Gbps best observed aggregate: about 2.0 up plus 1.9 down.

**0**
Adapter error/discard deltas in two documented counter checks.

## Observed environment and test record

- Windows 11 wired client using an Intel Ethernet Controller I225-V.
- eero edge and a Metronet symmetric 2 Gbps service path.
- Separate upload and reverse-download iPerf3 sessions ran concurrently.
- A documented receive run moved 33.5 GB in 150 seconds at about 1.92 Gbps average.
- Gateway and Internet PsPing windows were used to separate local from WAN queuing.
- Public endpoint performance varied; the strongest repeatable runs were retained as evidence.

## The correct conclusion - and its limit

**SUPPORTED CONCLUSION**

The observed path did not impose a 2 Gbps combined cap. It delivered near-line-rate traffic in both directions at the same time, consistent with full-duplex operation.

**NOT ESTABLISHED**

- Every public iPerf server can sustain the service rate.
- Every retransmission came from Amy's LAN or adapter.
- The same settings are optimal for another controller.
- The host, router, media, or ISP can sustain 5/5 or 10/10.

**PREREQUISITE**
Use the same evidence boundary when comparing future results.

**RESTART**
None.

**PASS**
Claims stay within the measured 2/2 path.

**FAIL**
A result is generalized to untested hardware or speed.

**RECOVERY**
Relabel the claim and run the appendix protocol.

PREPARATION

# Safety package before any change

PROVEN BASELINE

Preserve access, capture state, and change only one variable at a time.

## Before you touch a driver or adapter property

- [ ] Use a local administrator account; keep a second working sign-in if this PC is remote-managed.
- [ ] Schedule a short outage window. An adapter restart ends active calls, transfers, VPNs, and remote sessions.
- [ ] Download the intended OEM driver and confirm the exact device/OS match before removing anything.
- [ ] Keep a second connection or local console available when changing the primary adapter.
- [ ] Record the adapter name, hardware ID, driver version/date/provider, link speed, and every advanced property.
- [ ] Capture idle gateway and Internet latency plus adapter counter values.
- [ ] Name the symptom and the pass threshold before changing a setting.

## The one-change protocol

1. Capture the before state and save the exact command/output.
2. Apply one bounded change. Do not combine a driver, cable, MTU, and offload change.
3. Allow the adapter to reset. Reboot Windows only when the procedure or installer requires it.
4. Repeat the identical validation at least three times where throughput or latency varies.
5. Keep the change only when the predeclared pass criterion is repeatable.
6. If it fails, roll back first; collect new evidence only after basic connectivity returns.

A restore point can help with system configuration, but it is not a reliable substitute for an exported driver, written settings, or a second access path.

## Stop conditions

Stop immediately if the negotiated speed falls, the default gateway is unreachable, adapter errors/discards begin increasing, the PC becomes unreachable, or the correct rollback package is unavailable. Restore basic link state before continuing with performance tests.

Do not continue a stress test through overheating, driver resets, Event Viewer hardware errors, or sustained endpoint CPU saturation.

**PREREQUISITE**
Admin access, outage window, driver package, second path.

**RESTART**
Only when the specific change requires it.

**PASS**
One change is traceable to one result.

**FAIL**
Access or baseline evidence is missing.

**ROLLBACK**
Stop and restore connectivity before testing.

BASELINE

# Capture a reusable Windows baseline

PROVEN BASELINE

Run in PowerShell. The first block is non-destructive; the output folder is in Documents.

## 1 Create the capture folder and record the adapter

```
$stamp = Get-Date -Format "yyyyMMdd-HHmmss"
$dir = "$env:USERPROFILE\Documents\Network-Baseline-$stamp"
New-Item -ItemType Directory -Path $dir | Out-Null
Get-NetAdapter | Format-Table Name,InterfaceDescription,Status,LinkSpeed,MacAddress -Auto |
Out-File "$dir\adapters.txt"
Get-NetAdapterAdvancedProperty -Name "*" |
Format-Table Name,DisplayName,DisplayValue,RegistryKeyword,RegistryValue -Auto |
Out-File "$dir\advanced-properties.txt"
Get-NetAdapterStatistics -Name "*" | Format-List * |
Out-File "$dir\adapter-statistics.txt"
```

If the adapter alias is not Ethernet, substitute the exact value returned by Get-NetAdapter on later pages.

### Record before each test set

- ☐ Date, time, and test location
- ☐ Windows build and power plan
- ☐ Adapter alias and link speed
- ☐ Driver provider/version/date
- ☐ Router/SQM state
- ☐ Server name/IP and route
- ☐ iPerf3 version and command
- ☐ Idle gateway + Internet latency
- ☐ Endpoint CPU and temperature
- ☐ Counter deltas after the run

## 2 Capture routes, TCP state, RSS, IP configuration, and driver identity

```
Get-NetIPConfiguration | Format-List * | Out-File "$dir\ip-configuration.txt"
Get-NetIPInterface | Sort-Object AddressFamily,InterfaceMetric |
Format-Table ifIndex,InterfaceAlias,AddressFamily,AutomaticMetric,InterfaceMetric,NlMtu -Auto |
Out-File "$dir\interface-metrics.txt"
Get-NetAdapterRss -Name "*" | Format-List * | Out-File "$dir\rss.txt"
netsh interface tcp show global | Out-File "$dir\tcp-global.txt"
route print | Out-File "$dir\routes.txt"
Get-CimInstance Win32_PnPSignedDriver | Where-Object {$_.DeviceClass -eq "NET"} |
Select-Object DeviceName,DriverProviderName,DriverVersion,DriverDate,InfName |
Format-Table -Auto | Out-File "$dir\network-drivers.txt"
```

**PREREQUISITE**

PowerShell; administrator only for later mutations.

**RESTART**

None.

**VALIDATE**

Open every output file; confirm it is not empty.

**PASS**

Baseline can identify the exact adapter and restore values.

**IF IT FAILS**

Fix the alias/path; do not begin configuration work.

# Driver selection, install, and rollback

PROVEN BASELINE

Use the motherboard/system OEM package first; generic Intel packages are a fallback, not an automatic upgrade.

## Select and stage the package

1. Identify the exact controller and current INF from the baseline.
2. Check the computer or motherboard support page for the matching Windows release.
3. Compare the contained driver version, provider, compatible hardware IDs, and INF. A later download-page or package date is not proof of a newer device driver.
4. Read the release notes. Install only for a relevant fix, support need, or measured regression.
5. Use Intel's generic Windows package only when the OEM package is unavailable/inadequate and the Intel OS matrix includes the exact retail adapter.
6. Do not use third-party driver-updater utilities.

**OPTIONAL EXPORT**

```
pnputil /enum-drivers /class Net
pnputil /export-driver oem##.inf "$dir\DriverBackup"
```

Replace oem##.inf with the published name shown for the active package.

## Install without forcing an unnecessary reboot

**VENDOR INSTALLER**

1. Close network-dependent work and run the signed OEM installer as administrator.
2. Allow the adapter to reset. Do not interrupt a firmware component if the package explicitly includes one.
3. Restart Windows only if the installer requests it, the device remains in a pending state, or link behavior is inconsistent after the adapter reset.

**INF PACKAGE ALTERNATIVE**

```
pnputil /add-driver ".\DriverFolder\*.inf" /subdirs /install
```

PnPUtil selects the highest-ranked matching package; a numerically newer file is not automatically preferred.

**DO NOT**

Do not delete the old package or add /reboot until the replacement has been validated and local access is assured.

**RESTART**

Only if prompted, pending, or the adapter cannot recover.

**VALIDATE**

Driver identity, expected link speed, gateway, DNS, counters.

**PASS**

Correct package; stable link; no new error/discard deltas.

**FAIL**

Wrong speed, resets, errors, or loss of connectivity.

**ROLLBACK**

Device Manager > Driver > Roll Back, or reinstall exported/OEM INF.

ADAPTER HARDWARE

# Install or replace a PCIe Ethernet adapter safely

CONDITIONAL CHANGE

Use for 2.5, 5, or 10 GbE. Enumeration comes before driver or firmware work.

## Preflight before the shutdown

### IDENTIFY THE EXACT HARDWARE

1. Verify the complete retail/OEM adapter, supported Windows release, supplied bracket, and return path. A controller name on a marketplace listing is not proof of a supported retail card.
2. Read the exact system or motherboard revision manual. Confirm the slot's electrical width, generation, lane sharing, clearance, and airflow - not only connector length.
3. Stage the signed OEM/retail driver and rollback package. Keep the working onboard/old adapter enabled and its driver intact; leave the production cable there until the new card enumerates.
4. Check BitLocker or Device Encryption and retrieve the 48-digit recovery key from a second device or approved store before opening the case.

```
manage-bde -status C:
```

For a planned firmware, UEFI, option-ROM, or other measured-boot change, follow Microsoft/OEM suspend guidance and plan immediate resume. Do not disable TPM or Secure Boot as a shortcut.

5. Close work, perform a full shutdown, switch the PSU off, unplug AC power, and use ESD precautions. Do not hot-plug unless both the platform and adapter explicitly support it.

## Install, enumerate, then validate

### PHYSICAL AND ENUMERATION GATES

1. Use the correct bracket. Seat the card straight and evenly; no exposed contacts, PCB flex, case contact, modified metal, or force. Secure it snugly.
2. Leave the production cable on the known-good adapter. Power on and establish a normal firmware/Windows boot first.
3. In Device Manager and PowerShell, confirm a new PCIe/PnP network device exists before installing a driver or running a firmware updater.

```
Get-PnpDevice -Class Net | Sort-Object Status,FriendlyName
Get-NetAdapter | Format-Table Name,Status,LinkSpeed,InterfaceDescription -Auto
```

4. If absent, shut down and remove AC power. Inspect and reseat once, then try only a slot supported by the exact manual. If still absent, test/return the card; do not force firmware or arbitrary BIOS settings.
5. After enumeration, install the matching signed driver. Let the adapter reset. Restart Windows only if prompted, left pending, or the adapter does not recover cleanly.
6. Move the cable. Verify device/driver identity, negotiated speed, gateway, DNS, clean counters, a local throughput gate, one restart, and one cold start before retiring the old path.
7. Resume BitLocker protection if it was suspended and verify Protection On. Stop and use Microsoft/OEM recovery if entering the known key does not restore a normal boot.

### AMY'S BOUNDED FIELD RESULT

A later I226-based card install negotiated 2.5 Gbps and reached approximately 2 Gbps in Speedtest. Amy's later I226-T1 record showed device driver 2.1.5.7 dated 2025-09-03 inside Intel package 31.2.2; those are different version identities. The adapter reset and reconnected without a Windows reboot. These are bounded system/path records.

**PREREQUISITE**

Exact manual, signed driver, recovery key, old path.

**POWER**

Full shutdown and AC removal for hardware work.

**PASS**

Enumerated, correct link/driver, clean counters and boots.

**FAIL**

No PnP device, POST/power fault, recovery loop, or card flex.

**ROLLBACK**

Power off; remove card; restore old path; resume encryption.

# Evidence-backed Ethernet adapter baseline

PROVEN BASELINE

Property names vary by driver. Record the displayed value before changing anything.

## Recommended starting state

Speed & Duplex - Auto Negotiation. A forced speed can break negotiation and does not create bandwidth.

Receive Side Scaling (RSS) - Enabled. It distributes receive processing across available processors.

TCP/UDP checksum offloads - Enabled. Preserve hardware offload unless a documented defect is isolated.

Large Send Offload v2 - Enabled/default. Do not disable it merely because a tuning list says so.

Jumbo Packet / MTU - Standard Ethernet default. Every hop and endpoint must agree before an isolated-LAN experiment.

Interrupt Moderation - Driver default/adaptive. Lower latency can cost interrupt and CPU load.

Flow Control - Driver/OEM default. Do not force a state without a switch and workload reason.

Receive/Transmit buffers - Default. Increase only to test a CPU/burst/drop symptom.

Energy Efficient Ethernet - Default. Disable only for repeatable idle/resume or link-flap evidence.

Priority & VLAN - Disabled unless the network explicitly uses 802.1p/VLAN tagging.

## Exact change path

1. Open Device Manager.
2. Expand Network adapters.
3. Open the exact wired adapter.
4. Select Advanced.
5. Capture a screenshot or baseline export.
6. Change one property and select OK.
7. Allow the adapter to disconnect/reconnect.
8. Run the local validation block.

```
Get-NetAdapter -Name "Ethernet"
Get-NetAdapterRss -Name "Ethernet"
Get-NetAdapterStatistics -Name "Ethernet"
```

Use PowerShell mutation only after you have recorded the exact vendor-specific property/value names.

**RESTART**

Adapter reset is normal; Windows reboot usually unnecessary.

**VALIDATE**

Expected link speed, RSS enabled, gateway ping, counter deltas.

**PASS**

Stable link and equal/better repeated result.

**FAIL**

Negotiation falls, CPU spikes, drops/errors increase.

**ROLLBACK**

Restore the captured display value and reset the adapter.

# Latency and stability experiments

CONTROLLED EXPERIMENT

Each change needs a symptom, an A/B test, and a rollback. Neutral results return to baseline.

## Conditionally test - one at a time

**ENERGY EFFICIENT ETHERNET: OFF**

Only for repeatable idle/resume disconnects, renegotiation, or latency spikes that disappear when disabled.

**INTERRUPT MODERATION: LOW / OFF**

Only when low-latency packet handling matters and endpoint CPU/interrupt load remains acceptable.

**RECEIVE / TRANSMIT BUFFERS: HIGHER**

Only when counters or traces show drops under bursts and CPU/memory headroom exists.

**RSS QUEUES / PROCESSORS: HIGHER**

Only when receive CPU is concentrated and the driver/CPU topology supports more queues.

**FLOW CONTROL: ALTERNATE STATE**

Only with a known switch configuration and a specific loss/pausing hypothesis.

## Avoid cargo-cult tuning

- Disabling every checksum/segmentation offload.
- Forcing Speed & Duplex for ordinary production use.
- Enabling jumbo frames on an Internet path or on only one device.
- Applying registry TCP 'gaming' packs or undocumented timer changes.
- Maxing buffers, RSS queues, or interrupt rates without measuring CPU and counters.
- Changing five properties and treating one speed test as proof.
- Keeping a change because a synthetic benchmark improved while latency, stability, or normal workloads regressed.

A lower single ping is not sufficient. Compare the latency distribution, throughput, CPU load, adapter counters, and the real workload.

## Historical Amy configuration - field record, not a baseline

Before the successful Pro failover observation, Amy's recorded adapter state included Auto Negotiation and standard MTU; EEE off; Interrupt Moderation on with Low rate when Adaptive was unavailable; Flow Control off; ARP/NS, LSOv2, and TCP/UDP checksum offloads off; Priority/VLAN and unused wake features off; and Idle Power Down Restriction on when exposed. The bundle was not isolated or causally validated. Do not reproduce it wholesale; keep the defaults-first baseline and test only one justified change at a time.

**PREREQUISITE**

Named symptom + saved baseline + identical workload.

**RESTART**

Adapter reset usually enough; note the short outage.

**PASS**

Three-run result improves without a new regression.

**FAIL**

Neutral, inconsistent, hotter, higher CPU, or less stable.

**ROLLBACK**

Restore the one changed property immediately.

# Windows network stack and update behavior

CONDITIONAL CHANGE

Inspect first. Resets and update controls are recovery/maintenance tools, not speed tweaks.

## TCP state - preserve normal behavior

**INSPECT**

```
netsh interface tcp show global
Get-NetTCPSetting | Format-Table
SettingName,AutoTuningLevelLocal,ScalingHeuristics
```

Receive Window Auto-Tuning should normally remain Normal. Do not disable it to 'lower ping'; high-bandwidth, higher-latency paths need window growth.

**CONDITIONAL RESTORE**

```
netsh interface tcp set global autotuninglevel=normal
```

Use only if the inspection shows a non-normal state you did not intentionally set. Re-run the same test; no reboot is normally required.

**RECOVERY ONLY**

```
netsh winsock reset
netsh int ip reset "$dir\ip-reset.log"
```

These resets can remove custom networking state. They require a Windows restart and do not belong in routine performance tuning.

## Driver updates - choose one maintenance model

Leave Windows driver delivery enabled; install required Windows security/cumulative updates. Prefer the OEM network driver when a specific problem or support matrix calls for it.

**PRO / EDUCATION / ENTERPRISE - CONDITIONAL**

1. Stage a known-good OEM package, test rollback, and assign review of future driver/security releases.
2. Open gpedit.msc > Computer Configuration > Administrative Templates > Windows Components > Windows Update > Manage updates offered from Windows Update.
3. Open “Do not include drivers with Windows Updates,” select Enabled, then Apply > OK.

```
gpupdate /force
gpresult /scope computer /h "$env:USERPROFILE\Documents\network-policy.html"
```

Validate the report lists the enabled policy. No reboot is normally required. Rollback: set it Not Configured, Apply > OK, run gpupdate /force, and restore normal driver servicing.

**AFTER A BAD UPDATE**

Roll back the driver first; pause only long enough to prevent immediate reinstallation, document the version, then resume normal servicing.

**RESTART**

TCP reset: yes. Autotuning/policy inspection: no.

**VALIDATE**

TCP state, link, DNS, gateway, test repeatability.

**PASS**

Known state; updates still have an intentional owner/path.

**FAIL**

Reset removes required configuration or driver reappears.

**ROLLBACK**

Restore default servicing; reinstall known-good OEM driver.

HOST PATH PREFERENCE

# Deterministic primary and secondary interfaces

CONDITIONAL CHANGE

Interface metrics select new routes; they do not migrate an established TCP session.

## Set an explicit preference

### INSPECT BOTH ADDRESS FAMILIES

```
Get-NetIPInterface | Sort-Object AddressFamily,InterfaceMetric |
Format-Table
ifIndex,InterfaceAlias,AddressFamily,AutomaticMetric,InterfaceMetric,ConnectionState
-Auto
```

1. Choose the real primary and secondary adapter aliases.
2. Set the primary to a lower metric and the secondary to a higher metric.

```
Set-NetIPInterface -InterfaceAlias "Ethernet" -AddressFamily IPv4 -AutomaticMetric Disabled
-InterfaceMetric 10
Set-NetIPInterface -InterfaceAlias "Secondary" -AddressFamily IPv4 -AutomaticMetric Disabled
-InterfaceMetric 50
Set-NetIPInterface -InterfaceAlias "Ethernet" -AddressFamily IPv6 -AutomaticMetric Disabled
-InterfaceMetric 10
Set-NetIPInterface -InterfaceAlias "Secondary" -AddressFamily IPv6 -AutomaticMetric Disabled
-InterfaceMetric 50
```

Use values that preserve the ordering; the exact numbers have no inherent performance benefit.

## Validate route choice

```
Test-NetConnection 1.1.1.1 -DiagnoseRouting -InformationLevel Detailed
route print
```

- The selected outbound interface and source address are the primary.
- IPv4 and IPv6 behave as intended; otherwise one family may bypass the preference.
- DNS resolution works over each path when tested independently.
- No unintended default route has a lower combined route + interface metric.

### RESTORE AUTOMATIC METRICS

```
Set-NetIPInterface -InterfaceAlias "Ethernet" -AddressFamily IPv4
-AutomaticMetric Enabled
Set-NetIPInterface -InterfaceAlias "Secondary" -AddressFamily IPv4
-AutomaticMetric Enabled
```

Repeat for IPv6 if it was changed.

On Pro, metrics establish route preference; the next page adds the supported Connection Manager policy that holds Wi-Fi back while Ethernet exists.

**PREREQUISITE**

Both adapters already work independently.

**RESTART**

None.

**PASS**

New connections select the intended primary path.

**FAIL**

Wrong family/route, DNS failure, or asymmetric reachability.

**ROLLBACK**

Re-enable AutomaticMetric on every changed family.

# Failover validation - and its real boundary

CONDITIONAL CHANGE

Test planned recovery of new connections. A second adapter on the same upstream is not ISP redundancy.

## Controlled failover test

1. Confirm each interface works alone, Wi-Fi is allowed to connect automatically, and the metrics on page 11 are correct.
2. Start continuous gateway and Internet pings. Open a simple TCP test on the primary.
3. Disconnect the primary cable or disable only the primary adapter.

```
Disable-NetAdapter -Name "Ethernet" -Confirm:$false
```

4. Record loss duration, new source/interface, DNS behavior, and whether new connections succeed.
5. Restore the primary; verify the route returns only after the link is stable.

```
Enable-NetAdapter -Name "Ethernet" -Confirm:$false
```

Repeat the outage/failback cycle three times. Any failed cycle fails the validation.

Run locally if possible. Disabling the adapter ends a remote session that depends on it.

## Pro-only: Ethernet preferred, Wi-Fi held back

**WINDOWS 11 PRO / EDUCATION / ENTERPRISE**

1. Run gpedit.msc as administrator.
2. Go to Computer Configuration > Administrative Templates > Network > Windows Connection Manager.
3. Open “Minimize the number of simultaneous connections to the Internet or a Windows Domain.”
4. Set Enabled, then choose 3 = Prevent Wi-Fi when on Ethernet. Select OK.
5. Leave “Enable Windows to soft-disconnect...” Enabled or Not Configured so active traffic can settle.

```
gpupdate /force
```

The Wi-Fi radio can remain on. The policy prevents a WLAN connection while Ethernet exists; soft disconnect may preserve existing TCP traffic temporarily.

## Amy's Windows 11 Pro failover field record

**TWO PCS; ONE BOUNDED OBSERVATION**

Ethernet metric 10, Wi-Fi metric 50, WCM option 3, and a saved T-Mobile profile allowed to Auto-Connect.

- Cable removal moved both PCs to T-Mobile: one observed lost ping on I225-V/AX210; none observed on I226-V/Realtek 8851BE.
- Cable return restored Ethernet and left Wi-Fi disconnected; eero SQM behavior remained normal.

This is not an SLA or carrier-diverse enterprise failover. Existing sessions may reset, and both recovery layers share T-Mobile.

**RESTART**

None; adapter disable/enable creates the intended outage.

**PASS**

New connections use secondary within the declared objective.

**EXPECTED**

Existing TCP/VPN sessions may reset.

**FAIL**

No route/DNS, shared failure, or primary never returns cleanly.

**ROLLBACK**

Set policy Not Configured; gpupdate /force; restore metrics.

FIELD RESILIENCE

# Two tested failover layers, one T-Mobile backup

FIELD VERIFIED

Host-direct recovery and eero Internet Backup are complementary; both depend on the same backup service.

## Amy's field topology and edition boundary

**PRIMARY PATH**

Metronet symmetric 2/2 Gbps Internet -> eero gateway/network -> wired and Wi-Fi clients.

**FULL-NETWORK BACKUP - SUCCESSFUL FIELD TEST**

eero Plus Internet Backup is configured to use the T-Mobile Home Backup Internet Wi-Fi source. A whole-eero-network switch to that backup completed successfully.

**DIRECT HOST BACKUP**

The Windows 11 Pro PCs and M1 Pro MacBook Pro also have a direct T-Mobile backup path configured for host-level failover when their eero/Ethernet path is unavailable.

**CRITICAL BOUNDARY**

These are two control layers but only one backup carrier/gateway. A T-Mobile outage, allowance exhaustion, poor signal, or gateway power loss defeats both. This is not carrier-diverse backup.

When Ethernet remains linked and only the primary WAN fails, eero Internet Backup owns recovery. The Pro WCM policy is best matched to loss of the Ethernet path itself.

## Reproduce network-wide failover safely

1. Confirm eero Plus is active, the eero network is not in bridge mode, and the T-Mobile gateway is powered, in range, and has usable allowance.
2. In the eero app, open Security & privacy > eero Plus > eero Internet Backup. Add/select the T-Mobile backup network.
3. Start gateway/Internet pings and a lightweight new-connection check on at least one wired and one Wi-Fi eero client.
4. Disconnect only the primary WAN feed from the eero gateway. Do not power off the eero or T-Mobile gateway.
5. Record detection/recovery time, eero backup status, DNS, local-LAN access, public reachability, and representative workload behavior.
6. Restore the primary WAN and verify automatic failback after the path is stable.
7. Repeat the outage/failback cycle three times; one failed cycle fails validation.

**SCORE THE BACKUP FOR ITS JOB**

Existing sessions may reset and brief downtime is expected. Validate the minimum backup workload; do not run a multi-gigabit stress campaign against a metered/time-limited cellular backup.

**RESTART**

No reboot. Keep both gateways powered during the test.

**PASS**

eero clients recover on T-Mobile and automatically fail back.

**HOST PATH**

Direct T-Mobile fallback is tested separately per PC/Mac.

**FAIL**

Bridge mode, no signal/allowance, shared power, or no DNS.

**ROLLBACK**

Restore primary WAN; disable/remove backup network if needed.

# Install and verify the measurement toolkit

PROVEN BASELINE

Use a trusted iPerf3 build and Microsoft Sysinternals PsPing. Do not expose a test listener to the Internet.

## iPerf3 - throughput

1. Acquire iPerf3 from a trusted project/distribution source; preserve the version and hash used.
2. Extract it to a dedicated test folder and open PowerShell in that folder.

```
.\iperf3.exe --version
```

3. On a controlled server, listen only on the intended interface/firewall scope.

```
iperf3 -s -p 5201
```

4. Allow the test port only on the required private/test network. Never create a broad Internet port-forward.

Public servers were used in Amy's historical evidence, but a dedicated endpoint is the reproducible default. Follow every public server's rules and stop if it is busy.

## PsPing - latency distribution

1. Download PsPing/PsTools from Microsoft Sysinternals.
2. Run once to accept the license, or use the documented non-interactive switch.

```
.\psping64.exe -accepteula
.\psping64.exe -4 -n 10 -q 1.1.1.1
```

3. Keep separate terminals for gateway latency, Internet latency, upload, and download.

**CORE OPTIONS**

-4 IPv4 | -n sample count | -i interval | -q quiet | -h histogram | -R iPerf reverse | -P streams | -O omit warmup | -t duration

Confirm system time, server identity, commands, and router/SQM state in the test record.

**PREREQUISITE**

Trusted binaries; controlled server; scoped firewall.

**RESTART**

None.

**VALIDATE**

Version output, 10-ping smoke test, local listener.

**PASS**

Tools run and the test endpoint is attributable.

**ROLLBACK**

Remove the scoped firewall rule/listener after testing.

# Pass the local gate before testing the ISP

PROVEN BASELINE

The WAN cannot be interpreted until the client, server, NICs, slots, switch, and media work locally.

## Topology and local test sequence

- [ ] Draw every hop and record the negotiated rate at both ends of every link.
- [ ] Use a controlled server whose NIC, CPU, PCIe path, and switch port exceed the client/service target.
- [ ] Close VPNs, cloud sync, game downloads, backup jobs, and browser speed tests.

```
Get-NetAdapter | Format-Table Name,Status,LinkSpeed,InterfaceDescription -Auto
```

1. Run one TCP stream to reveal a per-flow/CPU/window limit.

```
.\iperf3.exe -c <LAN-SERVER-IP> -P 1 -t 30 -O 5
```

2. Run four streams, then eight only if throughput is still rising.

```
.\iperf3.exe -c <LAN-SERVER-IP> -P 4 -t 30 -O 5
.\iperf3.exe -c <LAN-SERVER-IP> -R -P 4 -t 30 -O 5
```

3. Repeat three times and compare counter deltas and endpoint CPU.

## Local acceptance gate

**PASS**

- Median application throughput is at least 90% of the slower negotiated link rate, or meets a documented platform-specific ceiling.
- No run is below 85% without an explained transient.
- Adapter error/discard deltas remain zero; link speed does not renegotiate.
- No endpoint core is saturated and no driver reset/thermal event occurs.

**FAIL - STAY LOCAL**

- Wrong link speed: inspect cable, port, negotiation, driver, and power state.
- One direction only is slow: swap endpoints/ports and inspect receive CPU/RSS.
- More streams plateau early: inspect CPU, PCIe width, server capacity, and switch path.

A LAN failure invalidates a WAN conclusion. Repair locally, then repeat the gate.

**RESTART**

None unless a driver/firmware correction explicitly needs it.

**PASS**

Three-run local gate clears target with clean counters.

**FAIL**

Do not call the ISP or tune TCP yet.

**ROLLBACK**

Return the last local change; swap one component.

**NEXT**

Proceed to single-direction WAN validation.

# Validate 2 Gbps one direction at a time

FIELD VERIFIED

Use a dedicated WAN endpoint when possible. Public servers are variable and may not permit sustained high-rate tests.

## Baseline latency first

**GATEWAY - SUBSTITUTE THE ACTUAL DEFAULT GATEWAY**

```
.\psping64.exe -4 -n 600 -i 0.1 -q -h 20 <GATEWAY-IP>
```

**INTERNET REFERENCE**

```
.\psping64.exe -4 -n 600 -i 0.1 -q -h 50 1.1.1.1
```

Record average, median/histogram shape, high-percentile behavior, loss, and isolated maxima. The distribution matters more than one outlier.

**BEFORE EACH THROUGHPUT SET**

- Save adapter counters.
- Confirm SQM state and that no other heavy workload is running.
- Record server, route, iPerf version, and endpoint CPU.

## Upload, reverse-download, repeat

**UPLOAD**

```
.\iperf3.exe -4 -c <SERVER> -p 5201 -P 8 -t 120 -O 10 --get-server-output
```

**DOWNLOAD**

```
.\iperf3.exe -4 -c <SERVER> -p 5201 -R -P 8 -t 120 -O 10
```

1. Run upload and download separately, at least three times each.
2. Use one stream, then four, before eight when characterizing a new endpoint.
3. Retest with a second controlled endpoint/time window if the result is inconsistent.

**2 GBPS METHOD GATE**

Pass: median at least 1.8 Gbps (90%), no run below 1.7 Gbps without explanation, clean local counters, stable plateau, and no endpoint CPU saturation.

**RESTART**

None.

**PASS**

Each direction independently clears the method gate.

**CONDITIONAL**

Remote endpoint/path varies; local gate remains clean.

**FAIL**

Repeatable deficit across controlled servers or local errors.

**NEXT**

Only then run simultaneous bidirectional load.

# Prove simultaneous bidirectional capacity

FIELD VERIFIED

Two independent sessions reduce the chance that one process, port, or remote server masks full-duplex capacity.

## Four-window run order

1. Start gateway and Internet PsPing. Wait about 10 seconds.

```
.\psping64.exe -4 -n 1400 -i 0.1 -q -h 20 <GATEWAY-IP>
.\psping64.exe -4 -n 1400 -i 0.1 -q -h 50 1.1.1.1
```

2. Start the upload and reverse-download sessions as close together as practical.

```
# Window A - upload
.\iperf3.exe -4 -c <UPLOAD-SERVER> -p 5201 -P 8 -t 150 -O 15 --get-server-output
# Window B - reverse download
.\iperf3.exe -4 -c <DOWNLOAD-SERVER> -p 5201 -R -P 8 -t 150 -O 15
```

3. Let every process finish. Record sender/receiver rates, retransmissions, CPU, latency, and adapter-counter deltas.
4. Repeat three times. A 16-stream escalation is acceptable only if the endpoint permits it and 8 streams have not reached a stable plateau.

## 2/2 simultaneous gate

**TARGET**

At least 1.8 Gbps in each direction and at least 3.6 Gbps aggregate application throughput.

**ALSO REQUIRED**

- Local errors/discards: zero delta.
- No link renegotiation or driver reset.
- Endpoint CPU not saturated.
- Gateway latency remains consistent with the chosen SQM/queueing state.

**DO NOT MISREAD**

Remote sender retransmissions indicate loss/congestion somewhere between that sender and the receiver. Clean local adapter counters argue against, but do not mathematically eliminate, a local cause.

A public server miss is not a service failure until controlled endpoints and the local gate agree.

**RESTART**

None.

**PASS**

Each direction >=1.8; aggregate >=3.6; counters clean.

**FAIL**

Same bounded deficit across controlled endpoints.

**ROLLBACK**

Stop load; restore SQM/adapter state used for baseline.

**RECORD**

Keep commands, interval output, latency, counters.

EVIDENCE RECORD

# Amy's approximately 3.8-3.9 Gbps validation

FIELD VERIFIED

The strongest simultaneous runs demonstrate the expected full-duplex aggregate for a symmetric 2/2 service.

**~ 3.8**

Gbps aggregate: approximately 1.9 Gbps upload + 1.9 Gbps download.

**~ 3.9**

Gbps best aggregate: approximately 2.0 Gbps upload + 1.9 Gbps download.

**33.5 GB**

Transferred in one 150-second receive run at approximately 1.92 Gbps.

## Corroborating observations

- Two captured post-test adapter checks showed zero received discards, received errors, outbound discards, and outbound errors.
- The successful simultaneous totals were far above a hypothetical 2 Gbps combined ceiling.
- The measured direction rates were close to the provisioned rate at the same time.
- Variation across public tests tracked endpoint/path limitations more than a persistent local ceiling.
- Heavy retransmissions on weak public runs were not accompanied by local adapter-error deltas.

## How to cite this result safely

**SAY**

Amy's recorded Windows/I225-V/eero/Metronet path sustained approximately 3.8-3.9 Gbps aggregate application throughput during concurrent upload and download testing.

**DO NOT SAY**

- The Ethernet link moved 4.0 Gbps of payload exactly.
- Every public test endpoint can reproduce the result.
- Zero adapter counters prove the entire path had zero loss.
- The result validates 5/5 or 10/10 equipment.

Application goodput, protocol overhead, sample timing, and rounded display values explain why this is an approximate record.

**RESTART**

None.

**PASS**

A future run stays within the same evidence boundary.

**FAIL**

Rounded totals or public servers are treated as guarantees.

**REPRODUCE**

Use pages 14-17 with a controlled endpoint.

**ESCALATE**

Use the appendix only after hardware/path prerequisites.

LATENCY UNDER LOAD

# eero SQM A/B test for loaded latency

CONTROLLED EXPERIMENT

SQM can reduce latency while reducing peak throughput. Test both states; bridge mode does not provide SQM.

## Exact A/B sequence

1. Open the eero app and tap Settings.
2. Tap Advanced networking.
3. Under Network services, tap SQM.
4. Record the current toggle state. No eero reboot is normally required; allow a few minutes after changing it.
5. Run three cycles in state A: idle, download-only, upload-only, then simultaneous bidirectional load.
6. Toggle SQM and repeat the identical server, commands, order, and time window for state B.
7. Compare throughput, gateway and Internet latency distributions, loss, counters, and application behavior.

eero documents that SQM is unavailable in bridge mode and may lower bandwidth for some devices while improving latency-sensitive work.

## Decision rule

**CALCULATE**

Added latency = loaded latency - idle latency. Compare medians and high-percentile behavior; do not optimize around one maximum.

**PRACTICAL INTERPRETATION - NOT A STANDARD**

0-5 ms exceptional | 5-15 very good | 15-30 good | 30-60 moderate | 60-100 significant | >100 severe

**KEEP SQM WHEN**

- Loaded latency improves materially and repeatedly.
- Throughput remains above the workload/service floor.
- The real application behaves better.

**ROLL BACK WHEN**

- The improvement is neutral/inconsistent or throughput cost is unacceptable.

Gateway stays low while Internet rises: likely WAN/upstream/remote queue. Both rise: investigate client/router/local queue.

**RESTART**

No reboot; allow the eero state to settle.

**PASS**

Repeatable loaded-latency gain above throughput floor.

**FAIL**

No gain, unstable result, or unacceptable throughput loss.

**ROLLBACK**

Return the toggle to its captured original state.

**RECORD**

Keep both states and all four workload results.

# Amy's eero radio record: healthy, no change

FIELD VERIFIED

Two wired eero Pro 7 nodes already had excellent 5/6 GHz conditions and no symptom that justified tuning.

## 2.4 GHz - busy air, quiet network

### REPRESENTATIVE NODE A SNAPSHOT

Total busyness 48% = 4% activity on this radio + 44% other-device interference. 24-hour average 36%; P99 47%; maximum 59%; noise floor -111 dBm; channel 6 at 20 MHz.

### LATER OBSERVATIONS ACROSS BOTH NODES

Averages were roughly 28-30%, current readings roughly 26-36%, and noise floor roughly -95 to -107 dBm. Node B sometimes heard more RF energy than Node A.

### DECISION

The eero network itself occupied little airtime; neighboring/other transmitters explained most of the busy time. With stable latency, no loss/disconnect symptom, and clean higher bands, no change was warranted.

Official eero definitions: total busyness is own-radio activity plus other-device interference; P99 describes the worst 1% more usefully than one maximum spike.

## 5 and 6 GHz - exceptionally clean

### 5 GHZ

Current busyness 4% = 1% own + 3% other; 24-hour average 5%; P99 9%; one transient maximum 69%; noise floor -105 dBm; channel 130 at 240 MHz.

### 6 GHZ

Across the wired Pro 7 nodes, average busyness was about 2%, P99 3%, maximum 4-5%, and noise floor about -107 dBm.

### CONTEXT

eero documents 240 MHz channel support on Wi-Fi 7 5/6 GHz radios. DFS operation and an isolated maximum are not failures by themselves; correlate a channel event with a real client symptom.

### EVIDENCE BOUNDARY

These values describe Amy's two-node, Ethernet-backhauled home at the observed times. They do not define universal pass thresholds, predict every client, or prove Wi-Fi can carry the wired 2/2 service rate.

**PREREQUISITE**

Compare 24-hour data and actual client symptoms.

**RESTART**

None; no setting changed.

**PASS**

Healthy clients + low sustained own activity + stable tests.

**FAIL**

A single maximum or neighbor traffic is treated as a fault.

**DECISION**

Preserve the working topology and automatic control.

# Optimize Wi-Fi only after a client fails a test

CONDITIONAL CHANGE

Observe -> localize -> change one physical or feature variable -> retest -> roll back if it does not help.

## Measurement sequence

1. Name the affected device, room, application, time window, and symptom. Keep a wired client as the control.
2. In the eero app: Home > select an eero > Wifi radio analytics. Choose 24 hours and record each node/band: average, P99, maximum, total, own activity, other interference, noise floor, channel, width, and channel-change times.
3. Run Settings > Troubleshooting > Device coverage scan. Treat an eero result of zero poor connections as healthy; follow placement guidance for a poor device.
4. From the same client position, ping the gateway and test both directions to a wired local server before testing the Internet.

```
.\iperf3.exe -c <WIRED-LAN-SERVER> -t 60 -O 5
.\iperf3.exe -c <WIRED-LAN-SERVER> -R -t 60 -O 5
```

5. Repeat three times on the same client, position, power state, band, and server. If aggregate capacity is the question, repeat separately with -P 4; do not hide the one-stream result.

Runbook consistency gate: no loss/abort, and each direction's slowest run is at least 85% of its three-run median. Choose a client-specific throughput target before testing.

Wi-Fi is shared, half-duplex airtime. Do not apply the wired 2/2, 5/5, or 10/10 gates to a wireless client; its radio, channel width, signal, contention, and power state may be the ceiling.

## Symptom-driven changes and stop rules

**IF OWN-RADIO ACTIVITY IS SUSTAINED AND THE CLIENT IS SLOW**

Identify local talkers, backups, cameras, or downloads; pause/reschedule one source and repeat the identical test.

**IF COVERAGE SCAN REPORTS A POOR CONNECTION**

Move the affected eero/client away from obstructions and retest. Add a node only when measured coverage remains poor and the app's guidance supports it; validate that contention and roaming do not worsen.

**IF OTHER INTERFERENCE IS HIGH BUT CLIENTS ARE HEALTHY**

Observe. eero chooses channels automatically and does not expose ordinary manual channel/width selection. Keep Ethernet backhaul and Client Steering defaults unless a documented compatibility symptom requires an A/B test.

**OPERATOR HEURISTICS - NOT EERO SPECIFICATIONS**

A high-performance client around -65 dBm or better, sustained busyness around 70-80%+, or noise approaching -90 dBm can justify closer investigation only when a repeatable symptom agrees. They are not automatic change thresholds.

Avoid repeated reboots to chase a channel: eero says long-term channel optimization takes roughly 2 days to 1 week, and resets restart that learning.

**RESTART**

Normally none; allow automatic optimization to settle.

**PASS**

No loss; >=85% consistency; declared client target met.

**FAIL**

No repeatable gain, new poor devices, or worse latency.

**ROLLBACK**

Restore prior placement/toggle; leave ACS automatic.

**ESCALATE**

Capture graphs, scan, client details, and exact times.

RESULT ANALYSIS

# Interpret the evidence without guessing

PROVEN BASELINE

A useful conclusion names the measurement point, direction, endpoint, and competing bottlenecks.

## Signal -> likely interpretation

Gateway latency rises under load - queueing is local to the client/router/LAN path or the router is saturated.

Gateway stays flat; Internet latency rises - WAN access, upstream, transit, or remote path queueing.

Only one remote server is slow - remote endpoint/route is the first suspect; retest elsewhere.

Download slow; upload normal - inspect reverse path, receive CPU/RSS, server sender, and service shaping by direction.

More streams help until a plateau - a single-flow window/CPU/path limit existed; plateau identifies the next ceiling.

Local adapter errors/discards increase - investigate physical link, driver, buffers, or hardware before blaming WAN.

Retransmissions rise but local counters stay clean - loss/congestion may be remote or in transit; still correlate timing and direction.

One endpoint core is pinned - the benchmark is CPU-limited even if total CPU appears low.

## Pass / conditional / fail language

**PASS**

Three-run median clears the predeclared gate; no unexplained low run; clean counters; stable negotiation; endpoints have headroom.

**CONDITIONAL PASS**

Local gate passes and a second controlled endpoint passes, while one public endpoint/path does not. Record the exception; do not tune the client around it.

**FAIL**

The same deficit repeats locally or across controlled WAN endpoints; errors/discards increase; negotiation falls; or endpoint/slot/CPU limits are demonstrated.

**INSUFFICIENT EVIDENCE**

One browser speed test, one public server, mixed Wi-Fi/wired tests, simultaneous unrelated traffic, unknown router state, or missing counter/CPU data.

Fast.com and browser speed tests can be useful corroboration, but their connection behavior and remote selection make them weak isolation tools.

**RESTART**

None.

**PASS**

Conclusion follows the same measurement in >=3 runs.

**FAIL**

Cause is assigned from correlation alone.

**RECOVERY**

Change server, localize the path, then repeat.

**EVIDENCE**

Keep raw commands/results; summarize only after.

# Failure decision tree

PROVEN BASELINE

Localize the first failing boundary; do not skip ahead to tuning.

**1 LINK** — Expected negotiation? → **2 LOCAL** — LAN >=90% and clean? → **3 SINGLE** — Each WAN direction? → **4 BOTH** — Simultaneous gate?

## If the first NO is at LINK or LOCAL

**WRONG NEGOTIATION / LINK FLAP**

- Try a known-good cable and port; verify both ends support the rate.
- Restore Auto Negotiation and documented driver defaults.
- Check driver identity, power/EEE symptoms, and event logs.

**LOCAL THROUGHPUT / COUNTERS**

- Swap endpoints and direction; inspect per-core CPU and RSS.
- Verify PCIe lane width, switch path, and server capacity.
- Change only one physical or driver variable, then repeat three runs.

Do not proceed to ISP tests until the local gate passes.

## If the first NO is at SINGLE or BOTH

**SINGLE DIRECTION**

- Use a second controlled endpoint and compare route/time.
- Confirm provisioning, router capacity, endpoint CPU, and SQM state.

**SIMULTANEOUS ONLY**

- Use separate servers/ports; check router full-duplex NAT/firewall/SQM capacity.
- Correlate gateway vs Internet latency and counter deltas.
- If each direction passes alone but both fail repeatedly, test router/handoff capacity before host tuning.

Escalate to the ISP with timestamps, local-gate proof, multiple controlled endpoints, commands, and direction-specific results.

**RESTART**
Only after a driver/firmware correction requires it.

**PASS**
The earliest failed boundary is identified.

**FAIL**
Multiple variables change before localization.

**ROLLBACK**
Return last variable and retest the same boundary.

**ESCALATE**
Send evidence, not a browser screenshot alone.

RECOVERY

# Rollback matrix and recovery order

PROVEN BASELINE

Restore basic connectivity first. Performance evidence is irrelevant while the path is unstable.

## Change -> immediate rollback -> reboot decision -> validation

Driver - Device Manager > adapter > Properties > Driver > Roll Back Driver, or reinstall the staged OEM/exported INF. Reboot only if prompted/pending. Validate driver identity, link, gateway, DNS, and counters.

Advanced property - restore the captured DisplayValue in Device Manager. Let the adapter reset; Windows reboot is rarely needed. Validate negotiation, RSS/offload state, gateway, and local gate.

Interface metric - re-enable AutomaticMetric for every changed address family. No reboot. Validate route selection and DNS over both paths.

SQM - return the eero toggle to its captured state and allow it to settle. No reboot. Repeat idle and one loaded-latency run.

TCP autotuning - restore the recorded state (normally Normal). No reboot unless a reset was also performed. Repeat the same one-stream and multi-stream test.

Winsock/IP reset - restore any required custom IP, DNS, proxy, VPN, binding, and route configuration from the baseline. Windows restart is required. Validate every business-critical path.

Failover test - re-enable the primary adapter, reconnect its cable, and confirm intended metrics/routes. No reboot. Expect applications to reconnect.

Jumbo/VLAN experiment - return every endpoint/switch hop to the prior consistent MTU/tagging state. Adapter/switch-port reset may be needed; validate standard ping and local throughput.

## Emergency recovery order

1 Stop the load -> 2 Re-enable/reconnect the known primary -> 3 restore automatic addressing/metrics if route choice is unclear -> 4 undo the last change -> 5 reboot only if the driver or network reset requires it -> 6 validate gateway, DNS, and local link -> 7 capture the failure before retrying.

**PASS**

Known-good connectivity and baseline state return.

**FAIL**

Link/DNS still broken after the last change is undone.

**NEXT**

Use local console/second path; reinstall known-good OEM driver.

**RESTART**

Only where the matrix says it is required.

**PRESERVE**

Do not delete the failed package/logs until analyzed.

CLOSEOUT

# Final acceptance record

PROVEN BASELINE

A result is complete only when the before state, command, pass gate, and rollback are reproducible.

## Environment and change record

- [ ] Date/time/time zone: _____
- [ ] Windows edition/build: _____
- [ ] Adapter/controller: _____
- [ ] Driver provider/version/date: _____
- [ ] Negotiated link / service target: _____
- [ ] Router / firmware / SQM state: _____
- [ ] Primary + secondary path: _____
- [ ] Change made (one only): _____
- [ ] Reason / named symptom: _____
- [ ] Rollback tested: YES / NO / NOT APPLICABLE
- [ ] Windows reboot actually required: YES / NO

## Measurement and decision

- [ ] Local median / lowest run: _____
- [ ] WAN upload median / lowest: _____
- [ ] WAN download median / lowest: _____
- [ ] Bidirectional upload + download: _____
- [ ] Aggregate application throughput: _____
- [ ] Idle / loaded gateway latency: _____
- [ ] Idle / loaded Internet latency: _____
- [ ] Error/discard deltas: _____
- [ ] Endpoint max core / temperature: _____
- [ ] Outcome: PASS / CONDITIONAL PASS / FAIL / ROLLED BACK
- [ ] Evidence folder / notes: _____

**ACCEPT**

Every required gate passes and rollback is known.

**CONDITIONAL**

Exception is bounded to server/path and recorded.

**REJECT**

Missing baseline, clean counters, or repeatability.

**INTEGRITY**

Record the release version and verify its checksum.

**PRIVACY**

Keep raw diagnostics and identifying evidence private.

SEPARATE SCALING APPENDIX

# 5/5 and 10/10 Gbps Stress-Test Method

Authoritative-documentation baseline

Not field-tested by Amy; not a product endorsement or line-rate guarantee

**DO NOT SCALE THE 2/2 SETTINGS BY MULTIPLICATION**

At 5 and 10 Gbps, endpoint CPU topology, PCIe lanes, switch fabric, router full-duplex capacity, media, driver support, thermals, and the test server become first-class limits. Clear each boundary locally before interpreting an Internet result.

AMY-UNTESTED

DOCUMENTATION-BACKED / CONTROLLED EXPERIMENT

# Architecture prerequisites for 5 and 10 Gbps

The slowest hop and the busiest shared resource set the result.

AMY-UNTESTED

DOCUMENTATION-BACKED / NOT FIELD-TESTED BY AMY

## Every boundary must exceed the target

- [ ] ISP handoff/ONT exposes the required per-direction rate and Ethernet mode.
- [ ] Router/firewall/NAT/SQM can sustain the simultaneous aggregate, not only one-way marketing throughput.
- [ ] Every switch port and internal fabric supports the rate without oversubscription on the tested path.
- [ ] Both endpoint NICs, drivers, OS releases, and negotiated modes match.
- [ ] PCIe slot is electrically wide/fast enough and not lane-shared into a lower mode.
- [ ] CPU has per-core and aggregate headroom; RSS is enabled and actually distributing flows.
- [ ] Media, connectors, modules/DAC, and length are supported at both ends.
- [ ] The test server has >= target-rate headroom and is not a public shared endpoint.
- [ ] Cooling/airflow keeps the NIC, PHY, chipset, and SFP+ modules within limits.
- [ ] No VPN, storage, browser, or security inspection bottleneck contaminates the path.

## Preflight commands and stop gates

```
Get-NetAdapter | Format-Table Name,Status,LinkSpeed,InterfaceDescription -Auto
Get-NetAdapterRss -Name "Ethernet" | Format-List
Enabled,NumberOfReceiveQueues,MaxProcessors,Profile
Get-NetAdapterStatistics -Name "Ethernet" | Format-List *
```

### HARDWARE CHECKS OUTSIDE WINDOWS

- Motherboard manual/block diagram: physical slot versus electrical lane width; GPU/M.2 sharing.
- Switch/router datasheet: port combinations, fabric/NAT/SQM performance, and thermal requirements.
- Vendor OS matrix: exact retail card/controller, driver branch, Windows release, and supported link modes.

### STOP

Do not start a WAN stress test if any hop negotiates below target, the local server lacks headroom, counters already increase, or the NIC/module is overheating.

A 10 Gb-capable motherboard label does not prove the onboard controller, driver, chipset path, router, or ISP handoff can sustain 10/10.

**RESTART**

Only after driver/firmware/slot changes require it.

**PASS**

Every hop negotiates at target with clean idle counters.

**FAIL**

Any shared resource or endpoint is below target.

**ROLLBACK**

Restore prior card/driver/slot and standard MTU.

**NEXT**

Clear the phased local throughput gates.

# Phased 5/5 and 10/10 throughput gates

AMY-UNTESTED

A publication method: median >=90% of negotiated rate, no unexplained run <85%, three runs, clean counters.

DOCUMENTATION-BACKED / NOT FIELD-TESTED BY AMY

## Stream progression and run order

1. Local test for 30 seconds; WAN test for 120 seconds after the local gate passes.
2. Begin with one TCP stream, then four, then eight. Stop adding streams when throughput plateaus.

```
.\iperf3.exe -c <SERVER> -P 1 -t 30 -O 5
.\iperf3.exe -c <SERVER> -P 4 -t 30 -O 5
.\iperf3.exe -c <SERVER> -P 8 -t 30 -O 5
.\iperf3.exe -c <SERVER> -R -P 4 -t 30 -O 5
```

3. Repeat the selected stream count three times in each direction.
4. Run simultaneous upload/reverse-download with two sessions or a controlled --bidir endpoint.
5. Record per-core CPU, negotiated speed, interval stability, retransmissions, and counter deltas.

iPerf3 3.16+ uses a separate thread per stream, but the endpoint, OS, and path can still bottleneck. Record the exact version.

## Application-throughput method gates

### 5 GBPS NEGOTIATED PATH

Single direction median >=4.5 Gbps. Simultaneous: each direction >=4.5 Gbps; aggregate >=9.0 Gbps.

### 10 GBPS NEGOTIATED PATH

Single direction median >=9.0 Gbps. Simultaneous: each direction >=9.0 Gbps; aggregate >=18.0 Gbps.

### ALSO REQUIRED

- No unexplained run below 85% of negotiated rate.
- Zero local error/discard deltas and no link renegotiation.
- No saturated endpoint core, PCIe bottleneck, thermal throttle, or driver reset.

### INTERPRET CAREFULLY

These are runbook acceptance gates, not universal service standards. Protocol overhead and implementation ceilings mean payload never equals the nominal PHY rate.

If four streams plateau but one does not, record both; do not hide a single-flow limitation behind stream count.

**RESTART**

None for testing.

**PASS**

Median >=90%; all runs >=85%; counters/CPU/thermals clean.

**FAIL**

Same deficit locally or across controlled endpoints.

**ROLLBACK**

Stop load; restore last driver/property/MTU change.

**ESCALATE**

Add streams only to characterize, never to conceal.

# Intel 10 GbE PCIe cards and onboard controllers

AMY-UNTESTED

Examples illustrate selection logic; verify the live Intel/OEM matrix before buying.

DOCUMENTATION-BACKED / NOT FIELD-TESTED BY AMY

## Documented Intel reference points

### X710-T2L - COPPER REFERENCE

Intel lists 10/5/2.5/1 GbE and 100 Mb per port, PCIe 3.0 x8, RJ45 media, and current Windows 11 client support in its retail-adapter matrix.

Use when the exact slot, cooling, OS/driver branch, and port-speed requirements match. The x8 requirement must be electrical, not merely a long connector.

### X550-T2 - CAUTIONARY REFERENCE

Intel lists dual 10 GbE, PCIe 3.0 x4, and current Windows 11 client support, but its product page states NBASE-T support is Linux-only. Do not infer Windows 2.5/5 Gb support from the controller's headline rates.

### X710-DA2 - SFP+ REFERENCE

Intel lists dual 10/1 GbE SFP+, PCIe 3.0 x8, and direct-attach twinax support up to 10 m. Module/cable/platform compatibility still must be verified.

These are capability references, not blanket purchase recommendations.

## Motherboard and card selection checklist

- [ ] Exact controller model/revision - not '2.5/5/10G LAN' marketing alone.
- [ ] Windows release and driver branch in the current OEM/Intel support matrix.
- [ ] Required link modes on Windows, including 2.5/5 Gb NBASE-T where needed.
- [ ] Physical and electrical slot width, generation, lane sharing, and GPU/M.2 impact.
- [ ] Full-height/low-profile bracket, port count, airflow, and heatsink clearance.
- [ ] OEM driver/firmware/BIOS dependencies for an onboard controller.
- [ ] Router/switch/handoff media and autonegotiation compatibility.
- [ ] Retail provenance and hardware revision; avoid unsupported/counterfeit server pulls.
- [ ] Return path and staged fallback NIC before changing the production adapter.

For onboard 10 GbE, read the motherboard block diagram: a chipset-attached port can share uplink bandwidth even when its PHY negotiates at 10 Gb.

**RESTART**

Card install/BIOS/driver package may require Windows restart.

**VALIDATE**

Device identity, lane width, driver, modes, thermals, local gate.

**PASS**

Exact OS/mode/slot/media combination is documented.

**FAIL**

Support is inferred from controller name or connector length.

**ROLLBACK**

Reinstall prior NIC/slot/driver and standard MTU.

# RJ45, SFP+, thermals, and experimental knobs

AMY-UNTESTED

Choose one supported media system end to end; adapter defaults remain the baseline.

DOCUMENTATION-BACKED / NOT FIELD-TESTED BY AMY

## Copper / RJ45 path

- For a new 10GBASE-T run, Cat6A is the conservative 100 m choice. Intel lists X550-T2 Cat6 to 55 m and Cat6A to 100 m.
- Intel lists X710-T2L RJ45 Category 5e/6/6A up to 100 m, but installation quality and alien crosstalk still matter.
- Use certified terminations and patch cords; avoid damaged, tightly bent, flat, or unknown-category cable.
- Verify both ends advertise the intended NBASE-T/10GBASE-T mode and link at the same rate.
- 10GBASE-T PHYs run warm. Provide chassis airflow and watch for renegotiation under a long full-duplex load.

**BASELINE**

Auto Negotiation, standard MTU, RSS/offloads enabled, driver-default interrupt moderation/buffers.

## SFP+ / DAC / fiber path

- Verify NIC, switch, module/DAC, firmware, and vendor compatibility matrix together.
- For DAC, verify passive/active type and supported length at both ends. Intel documents X710-DA2 twinax up to 10 m.
- For optics, match wavelength, fiber type, connector polish, reach, and transmit/receive pairing.
- Treat an SFP+ RJ45 module as a distinct thermal/compatibility case; many are distance- and platform-limited.
- Capture digital optical monitoring values when the platform exposes them; do not diagnose fiber from link-up alone.

**CONTROLLED EXPERIMENTS ONLY**

Jumbo frames, RSS-queue changes, interrupt moderation changes, buffer increases, and zero-copy/UDP tuning. Test one at a time on an isolated path and restore defaults if the declared metric does not improve.

**NEVER INFER**

A clean TCP test does not validate high-rate UDP; packet size, CPU, buffers, pacing, and MTU change that workload materially.

**RESTART**

Link/module reset first; reboot only for driver/firmware need.

**PASS**

Long full-duplex load holds rate, link, counters, and temperature.

**FAIL**

Renegotiation, errors, unsupported module, or thermal drift.

**ROLLBACK**

Known-good media; standard MTU; driver defaults.

**RECORD**

Exact cable/module/length/port/firmware combination.

REFERENCES

# Authoritative references - core runbook

PROVEN BASELINE

Reviewed 30 July 2026. Links are live; re-check vendor matrices and UI paths before use.


01 **ESnet - Invoking iPerf3**
software.es.net/iperf/invoking.html

02 **Microsoft Sysinternals - PsPing**
learn.microsoft.com/en-us/sysinternals/downloads/psping

03 **Microsoft Learn - Get-NetAdapterStatistics**
learn.microsoft.com/en-us/powershell/module/netadapter/get-netadapterstatistics

04 **Microsoft Learn - Get-NetIPInterface**
learn.microsoft.com/en-us/powershell/module/nettcpip/get-netipinterface

05 **Microsoft Learn - Test-NetConnection**
learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection

06 **Microsoft Learn - Windows Update client policies**
learn.microsoft.com/en-us/windows/deployment/update/waas-manage-updates-wufb

07 **Microsoft Learn - Windows Connection Manager policy (WCM)**
learn.microsoft.com/en-us/windows/client-management/mdm/policy-csp-admx-wcm

08 **Microsoft Learn - BitLocker recovery overview**
learn.microsoft.com/en-us/windows/security/operating-system-security/data-prote…

09 **Microsoft Learn - How Windows selects a driver package**
learn.microsoft.com/en-us/windows-hardware/drivers/install/how-windows-selects-…

10 **Microsoft Support - Update drivers through Device Manager**
support.microsoft.com/en-us/windows/update-drivers-through-device-manager-in-wi…

11 **Microsoft Learn - PnPUtil command syntax**
learn.microsoft.com/en-us/windows-hardware/drivers/devtest/pnputil-command-syntax

12 **Microsoft Learn - Network adapter performance tuning**
learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsys…

13 **Intel - Advanced settings for Intel Ethernet adapters**
www.intel.com/content/www/us/en/support/articles/000005593/ethernet-products.html

14 **Intel - Receive Side Scaling**
www.intel.com/content/www/us/en/support/articles/000006703/ethernet-products.html

15 **Intel - Interrupt Moderation Rate**
edc.intel.com/content/www/us/en/design/products/ethernet/adapters-and-devices-u…

16 **Intel - Windows 11 network adapter driver**
www.intel.com/content/www/us/en/download/727998/intel-network-adapter-driver-fo…

17 **eero - Smart Queue Management**
eero.com/support/articles/360000709886-What-is-Smart-Queue-Management-SQM

18 **eero - Internet Backup with eero Plus**
support.eero.com/hc/en-us/articles/8947860842523-What-is-eero-Internet-Backup

19 **eero - Wifi Radio Analytics**
support.eero.com/hc/en-us/articles/15282384606107-Wifi-Radio-Analytics

20 **eero - Coverage scan**
support.eero.com/hc/en-us/articles/34604365204635-What-is-Coverage-scan

21 **eero - How network optimization works**
support.eero.com/hc/en-us/articles/360057257311-How-Does-eero-Optimize-My-Network

22 **eero - Client Steering and DFS**
support.eero.com/hc/en-us/articles/207613326-What-advanced-features-does-eero-s…

23 **eero - Ethernet backhaul**
support.eero.com/hc/en-us/articles/207852743-Can-I-connect-my-eeros-with-Ethernet

24 **eero - Wi-Fi channel width**
support.eero.com/hc/en-us/articles/4888558892955-Channel-Size-and-Why-it-Matters

25 **T-Mobile - Home Internet Backup plan**
www.t-mobile.com/home-internet/plans/5g-backup-internet-options

26 **Flent - Introduction and RRUL**
flent.org/intro.html


**SOURCE-USE BOUNDARY**

Vendor defaults remain the baseline; server-focused material supplies architecture context, not a Windows 11 client mandate.

REFERENCES

# Authoritative references - adapter, SQM, and scale

AMY-UNTESTED

Current product examples are capabilities, not endorsements. The appendix remains outside Amy's field evidence.

DOCUMENTATION-BACKED / NOT FIELD-TESTED BY AMY


01 **ESnet - Network performance expectations**
fasterdata.es.net/performance-testing/troubleshooting/network-performance-expectations/

02 **ESnet - Test/measurement host tuning**
fasterdata.es.net/host-tuning/linux/test-measurement-host-tuning/

03 **ESnet - UDP tuning**
fasterdata.es.net/host-tuning/linux/udp-tuning/

04 **Intel - X710-T2L specifications**
www.intel.com/content/www/us/en/products/sku/189463/intel-ethernet-network-adapter-x710t2l/specifications.html

05 **Intel - Supported OS matrix for retail Ethernet adapters**
www.intel.com/content/www/us/en/support/articles/000025890/ethernet-products.html

01 **Intel - X550-T2 specifications**
www.intel.com/content/www/us/en/products/sku/88209/intel-ethernet-converged-network-adapter-x550t2/specifications.html

02 **Intel - X710-DA2 specifications**
www.intel.com/content/www/us/en/products/sku/83964/intel-ethernet-converged-network-adapter-x710da2/specifications.html

03 **Intel - Ethernet adapter installation guide**
edc.intel.com/content/www/us/en/design/products/ethernet/adapters-and-devices-user-guide/installation/

04 **Cisco - 10GBASE SFP+ modules data sheet**
www.cisco.com/c/en/us/products/collateral/interfaces-modules/transceiver-modules/data_sheet_c78-455693.html

05 **Cisco - Catalyst 9200 hardware guide (10GBASE-T media)**
www.cisco.com/c/en/us/td/docs/switches/lan/catalyst9200/hardware/install/b-c9200-hig/product_overview.pdf


**PUBLICATION CHECKPOINT**

Verify this PDF against outputs/SHA256SUMS.txt and use the same-version accessible Markdown companion.